Système d'exploitation : TD3 (*threads*, introduction)

février 2026 — Pierre Rousselin

Exercice 1 : Passer au type `void *(* )(void *)`

On considère le programme (idiot) suivant :

```c
#include <stdio.h>

int f(int a, int b);

int main(void)
{
        int n, m, r;
        printf("n : ");
        scanf("%d", &n);
        printf("m : ");
        scanf("%d", &m);

        r = f(n, m);
        printf("r = %d\n", r);

        return 0;
}

int f(int a, int b)
{
        return a * b - (a + b);
}
```

Le but du jeu est de modifier ce programme (sans changer sa logique), de manière à ce que la fonction utilisée ait pour prototype :

```c
void *f(void *arg);
```

On utilisera plusieurs méthodes :

1. les variables `n`, `m` et `r` sont globales, la valeur de retour de `f` est toujours `NULL`.
2. les nombres `n` et `m` sont stockés dans un tableau de deux `int` ; la fonction `f` alloue un entier sur le tas avec `malloc`, met le résultat de son calcul dedans et retourne l'adresse de cet entier ; `main` récupère la valeur calculée et libère la mémoire allouée par `f` ;
3. un type structuré contenant trois champs entiers : deux pour les arguments de `f` et un pour le résultat est déclaré. Une variable de ce type est déclarée et initialisée dans la pile de `main` et son adresse est transmise à `f` qui remplit le champs résultat et retourne `NULL`. Enfin, `main` récupère le résultat dans la structure.

--- * ---

Correction de l'exercice 1 :

1.
```c
#include <stdio.h>

void *f(void *arg);
int n, m, r;
int main(void)
{
        printf("n : ");
```

```
        scanf("%d", &n);
        printf("m : ");
        scanf("%d", &m);

        f(NULL);
        printf("r = %d\n", r);

        return 0;
}

void *f(void *arg)
{
        r = n * m - (n + m);
}
```

2.

```
#include <stdio.h>

void *f(void *arg);
int main(void)
{
        int *r, tab[2];
        printf("n : ");
        scanf("%d", tab);
        printf("m : ");
        scanf("%d", tab + 1);

        r = f(tab);
        printf("r = %d\n", *r);
        free(r);
        return 0;
}

void *f(void *arg)
{
        int *tab = arg;
        int *r = malloc(sizeof(int));
        *r = tab[0] * tab[1] - (tab[0] + tab[1]);
        return r;
}
```

3.

```
#include <stdio.h>

struct arg_f {
        int n, m, r;
};

void *f(void *arg);
int main(void)
{
        struct arg_f a;
        printf("n : ");
        scanf("%d", &(a.n));
        printf("m : ");
        scanf("%d", &(a.m));

        f(&a);
        printf("r = %d\n", a.r);

        return 0;
```

```
}

void *f(void *arg)
{
        struct arg_f *a = arg;
        a->r = a->n * a->m - (a->n + a->m);
        return NULL;
}
```

--- * ---

Exercice 2 : *threads*, exemples bons ou mauvais

Dans chacun des cas suivants, dire ce qu'affiche le programme, ou bien, s'il y a un comportement imprévisible ou des erreurs possibles à l'exécution, dire pourquoi. Pour des raisons de concision, les appels systèmes ne sont pas testés (mais en TP vous devez le faire).

1.
```
void *f(void *arg) {
        printf("hop\n");
        return NULL;
}
int main(void) {
        pthread_t p;
        pthread_create(&p, NULL, f, NULL);
        pthread_join(p, NULL);
        printf("plop\n");
        return 0;
}
```

2.
```
void *f(void *arg) {
        printf("hop\n");
        return NULL;
}
int main(void) {
        pthread_t p;
        pthread_create(&p, NULL, f, NULL);
        printf("plop\n");
        return 0;
}
```

3.
```
void *f(void *arg) {
        printf("hop\n");
        kill(getpid(), SIGTERM);
        return NULL;
}
int main(void) {
        pthread_t p;
        pthread_create(&p, NULL, f, NULL);
        printf("plop\n");
        return 0;
}
```

4.
```
void *f(void *arg) {
        printf("hop\n");
        strcpy(arg, "plop\n");
        return NULL;
}
int main(void) {
        pthread_t p;
        char ch[256];
        pthread_create(&p, NULL, f, ch);
```

```
        pthread_join(p, NULL);
        printf(ch);
        return 0;
}
```

5.
```
void *f(void *arg) {
        printf("hop\n");
        char *ch = malloc(256);
        strcpy(ch, "plop\n");
        return ch;
}
int main(void) {
        pthread_t p;
        void *ret;
        pthread_create(&p, NULL, f, NULL);
        pthread_join(p, &ret);
        printf(ret);
        free(ret);
        return 0;
}
```

6.
```
void *f(void *arg) {
        printf("hop\n");
        char ch[256];
        strcpy(ch, "plop\n");
        return ch;
}
int main(void) {
        pthread_t p;
        void *ret;
        pthread_create(&p, NULL, f, NULL);
        pthread_join(p, &ret);
        printf(ret);
        return 0;
}
```

--- * ---

Correction de l'exercice 2 :

1. L'affichage est `hop` suivi de `plop`. L'appel à `join` assure que le *thread* initial attend la fin du *thread* créé avant de lui-même afficher.

2. Comportement imprévisible. C'est l'échec, aucune synchronisation, tout peut arriver, y compris (et c'est ce qui arrivera le plus souvent) que le *thread* initial atteigne la fin de `main` et mette fin au processus, avant que le *thread* créé n'ai eu le temps d'afficher.

3. Comportement imprévisible. Le signal est envoyé à tout le processus, donc il est tout à fait possible que le *thread* créé mette fin au processus avant que le *thread* initial n'ait le temps d'afficher quoi que ce soit.

4. L'affichage est `hop` suivi de `plop`. Le *thread* créé a écrit dans la pile du *thread* initial la chaîne à afficher.

5. L'affichage est `hop` suivi de `plop`. Le *thread* créé a écrit dans le tas la chaîne `plop`. Le *thread* initial y a accédé puis a libéré la mémoire.

6. Comportement imprévisible. *Erreur courante !* Le *thread* créé a écrit dans sa pile, mais lorsqu'il se termine, sa pile est libérée. Le *thread* initial cherche à lire une chaîne dans une partie de la mémoire qui est libérée, ce qui provoquera sans doute une erreur de segmentation.

--- * ---

Exercice 3 : (Pris du partiel 1 2021 – 2022) Paralléliser, `void *` et pointeurs vers structures

On considère le programme suivant.

```
#define N 65536

void remplir_tab(unsigned tab[], size_t taille);
long unsigned somme_tab(unsigned tab[], size_t taille);
long unsigned produit_tab(unsigned tab[], size_t taille);
int main(void)
{
        unsigned tab[N];
        remplir_tab(tab, 10);
        printf("somme : %lu\n", somme_tab(tab, 10));
        printf("produit : %lu\n", produit_tab(tab, 10));
        return 0;
}
void remplir_tab(unsigned tab[], size_t taille) { /* ... */}
long unsigned somme_tab(unsigned tab[], size_t taille)
{
        long unsigned s = 0;
        size_t i;
        for (i = 0; i < taille; i = i + 1)
                s += tab[i];
        return s;
}
long unsigned produit_tab(unsigned tab[], size_t taille)
{
        long unsigned p = 1;
        size_t i;
        for (i = 0; i < taille; i = i + 1)
                p *= tab[i];
        return p;
}
```

La fonction

```
void remplir_tab(unsigned tab[], size_t taille);
```

initialise, d'une façon qui ne nous importe pas, les `taille` premières cases de `tab`.

On voudrait faire calculer la somme et le produit par 2 nouveaux *threads*.

1. Définir un type structuré `struct tableau` capable de contenir les informations nécessaires à ces *threads* pour calculer.
2. Définir la fonction

   ```
   /* Calcule le somme des éléments d'un tableau.
    * L'argument doit être l'adresse d'un élément du type struct tableau.
    * Retourne l'adresse d'un long unsigned alloué sur le tas contenant la
    * somme. */
   void *somme_tab_thread(void *arg);
   ```

   et la fonction `void *produit_tab_thread(void *arg)` de spécification similaire.
3. Écrire la fonction `main` correspondante (attention aux fuites de mémoire).

--- * ---

Correction de l'exercice 3 :

1.
```c
struct tableau {
        unsigned *tab;
        size_t taille;
};
```

2.
```c
void *somme_tab_thread(void *struct_tab)
{
        long unsigned *s = malloc(sizeof(long unsigned));
        struct tableau *t = struct_tab;
        *s = 0;
        size_t i;
        for (i = 0; i < t->taille; i = i + 1)
                *s += t->tab[i];
        return s;
}
void *produit_tab_thread(void *struct_tab)
{
        long unsigned *p = malloc(sizeof(long unsigned));
        struct tableau *t = struct_tab;
        *p = 1;
        size_t i;
        for (i = 0; i < t->taille; i = i + 1)
                *p *= t->tab[i];
        return p;
}
```

3.
```c
int main(void)
{
        unsigned tab[N];
        pthread_t th_s, th_p;
        remplir_tab(tab, 10);

        struct tableau t = { .tab = tab, .taille = 10 };
        pthread_create(&th_s, NULL, somme_tab_thread, &t);
        pthread_create(&th_p, NULL, produit_tab_thread, &t);
        void *ret;
        pthread_join(th_s, &ret);
        printf("somme : %lu\n", *((long unsigned *) ret));
        free(ret);
        pthread_join(th_p, &ret);
        printf("produit : %lu\n", *((long unsigned *) ret));
        free(ret);
        return 0;
}
```

--- * ---